CS301: Computability and Complexity Theory (CC)

Lecture 6: Reducibility

Dumitru Bogdan

Faculty of Computer Science
University of Bucharest

November 17, 2023

Table of contents

1. Previously on CS301
2. Context setup
3. Reduction
4. Computation history
5. Post Correspondence Problem
6. Mapping Reducibility
7. Turing Reducibility

Section 1

Previously on CS301

Some languages are not Turing-recognizable

Theorem

Some languages are not Turing-recognizable

This theorem has an important application to the theory of computation. It shows that some languages are not decidable or even Turing-recognizable, for the reason that there are uncountably many languages yet only countably many Turing machines. Because each Turing machine can recognize a single language and there are more languages than Turing machines, some languages are not recognized by any Turing machine.

Section 2

Context setup

Context setup

Corresponding to Sipser 5 & 6.3 & 6.4

Context setup

- We introduce the primary method for proving that problems are computationally unsolvable: **reducibility**
- We talk briefly about **computation history** and **Post Correspondence Problem**
- We continue studying the primary method for proving that problems are computationally unsolvable: **reducibility**
- Next, we delve into deeper aspects of computability theory: **turing reducibility** and **descriptive complexity**

Section 3

Reduction

Reduction

- A **reduction** is a way of converting one problem to another problem in such a way that a solution to the second problem can be used to solve the first problem. Such reducibilities come up often in everyday life, even if we don't usually refer to them in this way.
- For example, suppose that you want to find your way around a new city. You know that doing so would be easy if you had a map. Thus, you can reduce the problem of finding your way around the city to the problem of obtaining a map of the city
- Reducibility always involves two problems, which we call A and B. If A reduces to B, we can use a solution to B to solve A. So in our example, A is the problem of finding your way around the city and B is the problem of obtaining a map
- Note that reducibility says nothing about solving A or B alone, but only about the solvability of A in the presence of a solution to B

Reduction

- Reducibility also occurs in mathematical problems
- Measuring the area of a rectangle reduces to the problem of measuring its length and width
- Solving a system of linear equations reduces to the problem of inverting a matrix
- Reducibility plays an important role in classifying problems by decidability, and later in complexity theory as well
- When A is reducible to B, solving A cannot be harder than solving B because a solution to B gives a solution to A
- In terms of computability theory, if A is reducible to B and B is decidable, A also is decidable
- Equivalently, if A is undecidable and reducible to B, B is undecidable. This is key to proving that various problems are undecidable
- In short, our method for proving that a problem is undecidable will be to show that some other problem already known to be undecidable reduces to it

Halting problem

Let's consider $HALT_{TM}$, the problem of determining whether a TM halts (by accepting or rejecting) on a given input. This problem is widely known as the **halting problem**. We use the undecidability of A_{TM} to prove the undecidability of the halting problem by reducing A_{TM} to $HALT_{TM}$. Let

$$HALT_{TM} = \{< M, w > | M \text{ is a TM and } M \text{ halts on input } w\}$$

Theorem

$HALT_{TM}$ is undecidable

$HALT_{TM}$ is undecidable

Proof idea: This proof is by contradiction. We assume that $HALT_{TM}$ is decidable and use that assumption to show that A_{TM} is decidable which is not true. The key idea is to show that A_{TM} is reducible to $HALT_{TM}$. Let's assume that we have a TM R that decides $HALT_{TM}$. Then we use R to construct S, a TM that decides A_{TM} . To get a feel for the way to construct S, pretend that you are S. Your task is to decide A_{TM} . You are given an input of the form $< M, w >$. You must output **accept** if M accepts w, and you must output **reject** if M loops or rejects on w. Try simulating M on w. If it accepts or rejects, do the same. But you may not be able to determine whether M is looping, and in that case your simulation will not terminate. That's bad because you are a decider and thus never permitted to loop. So this idea by itself does not work.

$HALT_{TM}$ is undecidable

Instead, use the assumption that you have TM R that decides $HALT_{TM}$. With R, you can test whether M halts on w. If R indicates that M doesn't halt on w, reject because $< M, w >$ isn't in A_{TM}. However, if R indicates that M does halt on w, you can do the simulation without any danger of looping.

Thus, if TM R exists, we can decide A_{TM}, but we know that A_{TM} is undecidable. By virtue of this contradiction, we can conclude that R does not exist. Therefore, $HALT_{TM}$ is undecidable.

$HALT_{TM}$ is undecidable

Proof.

Let's assume for the purpose of obtaining a contradiction that TM R decides $HALT_{TM}$. We construct TM S to decide A_{TM} , with S operating as follows.

$S =$ On input $< M, w >$, an encoding of a TM M and a string w

1. Run TM R on input $< M, w >$
2. If R rejects, *reject*
3. If R accepts, simulate M on w until it halts
4. If M has accepted, *accept*; if M has rejected, *reject*

Clearly, if R decides $HALT_{TM}$, then S decides A_{TM} . Because A_{TM} is undecidable, $HALT_{TM}$ also must be undecidable. □

E_{TM} is undecidable

Previous theorem illustrates our strategy for proving that a problem is undecidable. This strategy is common to most proofs of undecidability, except for the undecidability of A_{TM} itself, which is proved directly via the diagonalization method.

We now study several other theorems and their proofs as further examples of the reducibility method for proving undecidability. Let

$$E_{TM} = \{< M > | M \text{ is a TM and } L(M) = \emptyset\}$$

Theorem

E_{TM} is undecidable

E_{TM} is undecidable

Proof idea: We assume that E_{TM} is decidable and then show that A_{TM} is decidable getting a contradiction. Let R be a TM that decides E_{TM}. We use R to construct TM S that decides A_{TM}. How will S work when it receives input $< M, w >$? One idea is for S to run R on input $< M >$ and see whether it accepts. If it does, we know that $L(M)$ is empty and therefore that M does not accept w. But if R rejects $< M >$, all we know is that $L(M)$ is not empty and therefore that M accepts some string, however we still do not know whether M accepts the particular string w. So we need to use a different idea.

Instead of running R on $< M >$, we run R on a modification of $< M >$. We modify $< M >$ to guarantee that M rejects all strings except w, but on input w it works as usual. Then we use R to determine whether the modified machine recognizes the empty language. The only string the machine can now accept is w, so its language will be nonempty iff it accepts w. If R accepts when it is fed a description of the modified machine, we know that the modified machine doesn't accept anything and that M doesn't accept w.

E_{TM} is undecidable

Proof.

Let's create the modified machine described in the proof idea using our standard notation. We call it M_1

M_1 = On input x:

1. if $x \neq w$, *reject*
2. if $x = w$, run M on input w and *accept* if M does

This machine has the string w as part of its description. It conducts the test of whether $x = w$ in the obvious way, by scanning the input and comparing it character by character with w to determine whether they are the same. Putting all this together, we assume that TM R decides E_{TM} and construct TM S that decides A_{TM} as follows

E_{TM} is undecidable

Proof.

S = In input $< M, w >$, an encoding of a TM M and a string w:

1. Use the description of M and w to construct the TM M_1 just described
2. run R in input $< M_1 >$
3. if R accepts, *reject*; if R rejects, *accept*

S is able to compute a description of M_1 from a description of M and w. It is able to do so because it only needs to add extra states to M that perform the $x = w$ test. If R were a decider for E_{TM}, S would be a decider for A_{TM}. A decider for A_{TM} cannot exist, so we know that E_{TM} must be undecidable. □

$REGULAR_{TM}$ is undecidable

Another interesting computational problem regarding TM concerns determining whether a given TM recognizes a language that also can be recognized by a simpler computational model. For example, we let $REGULAR_{TM}$ be the problem of determining whether a given TM has an equivalent finite automaton. This problem is the same as determining whether the TM recognizes a regular language. Let

$$REGULAR_{TM} = \{< M > | M \text{ is a TM and } L(M) \text{ is a regular language}\}$$

Theorem

$REGULAR_{TM}$ *is undecidable*

$REGULAR_{TM}$ is undecidable

Proof idea: This proof is by reduction from A_{TM} . We assume that $REGULAR_{TM}$ is decidable by a TM R and use this assumption to construct a TM S that decides A_{TM} . Less obvious now is how to use R's ability to assist S in its task.

The idea is for S to take its input $< M, w >$ and modify M so that the resulting TM recognizes a regular language if and only if M accepts w. We call the modified machine M_2 . We design M_2 to recognize the nonregular language $\{0^n1^n | n \geq 0\}$ if M does not accept w, and to recognize the regular language Σ^* if M accepts w. We must specify how S can construct such an M_2 from M and w. Here, M_2 works by automatically accepting all strings in $\{0^n1^n | n \geq 0\}$. In addition, if M accepts w, M_2 accepts all other strings.

We construct M_2 only for the purpose of feeding its description into the decider for $REGULAR_{TM}$ that we have assumed to exist. Once this decider returns its answer, we can use it to obtain the answer to whether M accepts w. Thus, we can decide A_{TM}, a contradiction.

$REGULAR_{TM}$ is undecidable

Proof.

S = On input $< M, w >$, where M is a TM and w is a string:

1. Construct the following TM M_2
   M_2 = On input x:
   1. If x has the form 0^n1^n, *accept*
   2. if x does not have this form, run M on input w and *accept* if M accepts w
2. Run R in input $< M_2 >$
3. If R accepts, *accept*; if R rejects, *reject*

□

Rice's theorem

Similarly, the problems of testing whether the language of a TM is a context-free language, a decidable language, or even a finite language can be shown to be undecidable with similar proofs. In fact, a general result, called **Rice's theorem**, states that determining any property of the languages recognized by TM is undecidable.

EQ_{TM} is undecidable

So far, our strategy for proving languages undecidable involves a reduction from A_{TM}.
Sometimes reducing from some other undecidable language, such as E_{TM} , is more convenient when we are showing that certain languages are undecidable.
We could prove it by a reduction from A_{TM}, but we use this opportunity to give an example of an undecidability proof by reduction from E_{TM}. Let

$$EQ_{TM} = \{< M_1, M_2 > | M_1 \text{ and } M_2 \text{ are TMs and } L(M_1) = L(M_2)\}$$

Theorem

EQ_{TM} is undecidable

EQ_{TM} is undecidable

Proof idea: Show that if EQ_{TM} were decidable, E_{TM} also would be decidable by giving a reduction from E_{TM} to EQ_{TM}. The idea is simple. E_{TM} is the problem of determining whether the language of a TM is empty. EQ_{TM} is the problem of determining whether the languages of two TMs are the same. If one of these languages happens to be $\emptyset$, we end up with the problem of determining whether the language of the other machine is empty—that is, the E_{TM} problem. So in a sense, the E_{TM} problem is a special case of the EQ_{TM} problem wherein one of the machines is fixed to recognize the empty language. This idea makes giving the reduction easy.

EQ_{TM} is undecidable

Proof.

We let TM R decide EQ_{TM} and construct TM S to decide E_{TM} as follows

$S =$ On input $< M >$, where M is a TM:

1. Run R on input $< M, M_1 >$, where M_1 is a TM that rejects all inputs.
2. it R accepts, *accept*; it R rejects, *reject*

□

If R decides EQ_{TM} , S decides E_{TM}. But E_{TM} is undecidable, so EQ_{TM} also must be undecidable.

Section 4

Computation history

Computation history

The computation history method is an important technique for proving that A_{TM} is reducible to certain languages. This method is often useful when the problem to be shown undecidable involves testing for the existence of something. For example, this method is used to show the undecidability of Hilbert's tenth problem, testing for the existence of integral roots in a polynomial. The computation history for a Turing machine on an input is simply the sequence of configurations that the machine goes through as it processes the input. It is a complete record of the computation of this machine.

Computation history

Definition

Let M be a Turing machine and w an input string. An *accepting computation history* for M on w is a sequence of configurations, $C_1, C_2, \ldots, C_l$, where C_1 is the start configuration of M on w, C_l is an accepting configuration of M, and each C_i legally follows from C_{i-1} according to the rules of M. A *rejecting computation history* for M on w is defined similarly, except that C_l is a rejecting configuration.

Computation histories are finite sequences. If M doesn't halt on w, no accepting or rejecting computation history exists for M on w. Deterministic machines have at most one computation history on any given input. Nondeterministic machines may have many computation histories on a single input, corresponding to the various computation branches.

Computation history

Our first undecidability proof using the computation history method concerns a type of machine called a linear bounded automaton

Definition

A **linear bounded automaton** is a restricted type of TM wherein the tape head isn't permitted to move off the portion of the tape containing the input. If the machine tries to move its head off either end of the input, the head stays where it is—in the same way that the head will not move off the left-hand end of an ordinary TM's tape.

A linear bounded automaton is a TM with a limited amount of memory. It can only solve problems requiring memory that can fit within the tape used for the input. Using a tape alphabet larger than the input alphabet allows the available memory to be increased up to a constant factor. Hence we say that for an input of length n, the amount of memory available is linear in n thus the name of this model

A_{LBA} is decidable

Despite their memory constraint, linear bounded automata (LBAs) are quite powerful. For example, the deciders for A_{DFA} , A_{CFG} , E_{DFA} , and E_{CFG} all are LBAs. Let

$$A_{LBA} = \{< M, w > \mid M \textit{ is an LBA that accepts string } w\}$$

Theorem

A_{LBA} is decidable

A_{LBA} is decidable

Lemma

Let M be an LBA with q states and g symbols in the tape alphabet. There are exactly qng^n distinct configurations of M for a tape of length n

Proof.

Recall that a configuration of M is like a snapshot in the middle of its computation. A configuration consists of the state of the control, position of the head, and contents of the tape. Here, M has q states. The length of its tape is n, so the head can be in one of n positions, and g^n possible strings of tape symbols appear on the tape. The product of these three quantities is the total number of different configurations of M with a tape of length n. □

A_{LBA} is decidable

Proof idea: In order to decide whether LBA M accepts input w, we simulate M on w. During the course of the simulation, if M halts and accepts or rejects, we accept or reject accordingly. The difficulty occurs if M loops on w. We need to be able to detect looping so that we can halt and reject.

The idea for detecting when M is looping is that as M computes on w, it goes from configuration to configuration. If M ever repeats a configuration, it would go on to repeat this configuration over and over again and thus be in a loop. Because M is an LBA, the amount of tape available to it is limited. By previous lemma, M can be in only a limited number of configurations on this amount of tape. Therefore, only a limited amount of time is available to M before it will enter some configuration that it has previously entered. Detecting that M is looping is possible by simulating M for the number of steps given by previous lemma. If M has not halted by then, it must be looping.

A_{LBA} is decidable

Theorem

A_{LBA} is decidable

Proof.

The algorithm that decides A_{LBA} is as follows:

L = On ipunt $< M, w >$, where M is a LBA and w is a string:

1. Simulate M on w for qng^n steps or until it halts
2. If M has halted; *accept* if it has accepted or *reject* if it has rejected. If has not halted, *reject*

If M on w has not halted within qng^n steps, it must be repeating a configuration according to previous lemma and therefore looping. That is why our algorithm rejects in this instance. □

Section 5

Post Correspondence Problem

PCP is undecidable

Next, we show that the phenomenon of undecidability is not confined to problems concerning automata. We study a problem that can easily be described as a puzzle and it is called **Post Correspondence Problem** (PCP).
We begin with a collection of dominos, each containing two strings, one on each side. An individual domino looks like

$$\left[\frac{a}{b}\right]$$

and a collection of dominos looks like

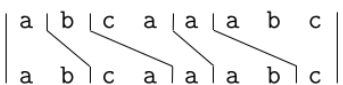
$$\left\{\left[\frac{b}{ca}\right], \left[\frac{a}{ab}\right], \left[\frac{ca}{a}\right], \left[\frac{abc}{c}\right]\right\}$$

PCP is undecidable

The task is to make a list of these dominos (repetitions permitted) so that the string we get by reading off the symbols on the top is the same as the string of symbols on the bottom. This list is called a *match*. For example, the following list is a match for this puzzle

$$\left[\frac{a}{ab}\right], \left[\frac{b}{ca}\right], \left[\frac{ca}{a}\right], \left[\frac{a}{ab}\right], \left[\frac{abc}{c}\right]$$

Reading off the top string we get *abcaaabc*, which is the same as reading off the bottom. We can also depict this match by deforming the dominos so that the corresponding symbols from top and bottom line up



PCP is undecidable

For some collections of dominos, finding a match may not be possible. For example, the collection

$$\left\{\left[\frac{abc}{ab}\right], \left[\frac{ca}{a}\right], \left[\frac{acc}{ba}\right]\right\}$$

cannot contain a match because every top string is longer than the corresponding bottom string.

PCP is undecidable

We state the problem precisely and then express it as a language. An instance of the PCP is a collection P of dominos

$$P = \left\{ \left[\frac{t_1}{b_1}\right], \left[\frac{t_2}{b_2}\right], \ldots, \left[\frac{t_k}{b_k}\right] \right\}$$

and a match is a sequence $i_1, i_2, \ldots, i_l$, where $t_{i_1} t_{i_2} \ldots t_{i_l} = b_{i_1} b_{i_1} \ldots t_{i_l}$. The problem is to determine if P has a match. Let

$$PCP = \{< P > | P \textit{ is an instance of the Post Correspondence Problem with a match}\}$$

Theorem

PCP is undecidable

PCP is undecidable

Proof idea: Conceptually this proof is simple, though it involves many details. The main technique is reduction from A_{TM} via accepting computation histories. We show that from any TM M and input w, we can construct an instance P where a match is an accepting computation history for M on w. If we could determine whether the instance has a match, we would be able to determine whether M accepts w.
How can we construct P so that a match is an accepting computation history for M on w? We choose the dominos in P so that making a match forces a simulation of M to occur. In the match, each domino links a position or positions in one configuration with the corresponding one(s) in the next configuration.

Proof.

homework :) □

Section 6

Mapping Reducibility

Mapping Reducibility

- We have shown how to use the reducibility technique to prove that various problems are undecidable.
- Next, we formalize the notion of reducibility. Doing so allows us to use reducibility in more refined ways, such as for proving that certain languages are not Turing-recognizable and for applications in complexity theory.
- We concentrate on a simple type of reducibility called **mapping reducibility**
- Being able to reduce problem A to problem B by using a mapping reducibility means that a computable function exists that converts instances of problem A to instances of problem B
- If we have such a conversion function, called a *reduction*, we can solve A with a solver for B
- The reason is that any instance of A can be solved by first using the reduction to convert it to an instance of B and then applying the solver for B

Computable functions

A TM computes a function by starting with the input to the function on the tape and halting with the output of the function on the tape

Definition

A function $f : \Sigma^* \to \Sigma^*$ is a **computable function** if some TM M, on every input w, halts with just $f(w)$ on its tape.

Example: All usual, arithmetic operations on integers are computable functions. For example, we can make a machine that takes input $< m, n >$ and returns $m + n$, the sum of m and n.

Mapping Reducibility

Next, we define mapping reducibility. As usual, we represent computational problems by languages.

Definition

Language A is mapping reducible to language B, written $A \leq_m B$ if there is a computable function $f : \Sigma^* \to \Sigma^*$, where for every w

$$w \in A \iff f(w) \in B$$

the function f is called the **reduction** from A to B.

Mapping Reducibility

A mapping reduction of A to B provides a way to convert questions about membership testing in A to membership testing in B. To test whether $w \in A$, we use the reduction f to map w to $f(w)$ and test whether $f(w) \in B$. The term mapping reduction comes from the function or mapping that provides the means of doing the reduction.

If one problem is mapping reducible to a second, previously solved problem, we can thereby obtain a solution to the original problem. We capture this idea in next theorem

Theorem

If $A \leq_m B$ and B is decidable, then A is decidable.

Mapping Reducibility

Proof.

We let M be the decider for B and f be the reduction from A to B. We describe a decider N for A as follows.

$N =$ On input w:

1. Compute $f(w)$
2. Run M on input $f(w)$ and output whatever M outputs

Clearly, if $w \in A$, then $f(w) \in B$ because f is a reduction from A to B. Thus, M accepts $f(w)$ whenever $w \in A$. Therefore, N works as desired. □

Corollary

If $A \leq_m B$ and B is undecidable, then B is undecidable.

Example I

We used a reduction from A_{TM} to prove that $HALT_{TM}$ is undecidable. This reduction showed how a decider for $HALT_{TM}$ could be used to give a decider for A_{TM}. We can demonstrate a mapping reducibility from A_{TM} to $HALT_{TM}$ as follows. To do so, we must present a computable function f that takes input on the form $< M, w >$ and returns putput $< M', w' >$, where

$$< M, w > \in A_{TM} \iff < M', w' > \in HALT_{TM}$$

Example I

The following machine F computes a reduction f

F = On iput $< M, w >$:

1. Construct the following machine M'
   M' = On input x:
   1. Run M on x
   2. If M accepts, *accept*
   3. If M rejects, enter a loop
2. Output $< M', w >$

A minor issue arises here concerning improperly formed input strings. If TM F determines that its input is not of the correct form as specified in the input line "On input $< M, w >$:" and hence that the input is not in A_{TM}, the TM outputs a string not in $HALT_{TM}$. Any string not in HALT TM will do. In general, when we describe a TM that computes a reduction from A to B, improperly formed inputs are assumed to map to strings outside of B.

Example II

A mapping reduction from E_{TM} to EQ_{TM} is function f that maps the input $< M >$ to the output $< M, M_1 >$, where M_1 is the machine that rejects all inputs.

The proof of theorem showing that E_{TM} is undecidable illustrates the difference between the formal notion of mapping reducibility that we have defined in this section and the informal notion of reducibility that we used earlier. The proof shows that E_{TM} is undecidable by reducing A_{TM} to it. Let's see whether we can convert this reduction to a mapping reduction. From the original reduction, we may easily construct a function f that takes input $< M, w >$ and produces output M_1, where M_1 is the TM described in that proof. But M accepts w iff $L(M_1)$ is not empty so f is a mapping reduction from A_{TM} to E_{TM}. It still shows that E_{TM} is undecidable because decidability is not affected by complementation, but it doesn't give a mapping reduction from A_{TM} to E_{TM}. In fact, no such reduction exists.

Section 7

Turing Reducibility

Turing Reducibility

- We already introduced the reducibility as a way of using a solution to one problem to solve other problems
- Thus, if A is reducible to B, and we find a solution to B, we can obtain a solution to A
- Subsequently, we described mapping reducibility, a specific form of reducibility
- But does mapping reducibility capture our intuitive concept of reducibility in a more general way?
- Let us consider A_{TM} and $\overline{A_{TM}}$
- Intuitively, they are reducible to one another because a solution to either could be used to solve the other by simply reversing the answer
- However, we know that $\overline{A_{TM}}$ is not mapping reducible to A_{TM} because A_{TM} is Turing-recognizable but $\overline{A_{TM}}$ is not
- We present a very general form of reducibility called **Turing reducibility**

Oracle TMs

Definition

An **oracle** for a language B is an external device that is capable of reporting whether any string w is a member of B. An **oracle TM** is a modified TM that has the additional capability of querying an oracle. We write M^B to describe an oracle TM that has an oracle for language B.

We aren't concerned with the way the oracle determines its responses. We use the term oracle to connote a magical ability and consider oracles for languages that aren't decidable by ordinary algorithms.

Example I

Consider an oracle for A_{TM}. An oracle TM with an oracle for A_{TM} can decide more languages than an ordinary TM can. Such a machine can (obviously) decide A_{TM} itself, by querying the oracle about the input. It can also decide E_{TM}, the emptiness testing problem for TMs with the following procedure called $T^{A_{TM}}$.

$T^{A_{TM}}$ = On input $< M >$, where M is a TM:

1. Construct the following TM N
   N = On any input:
   1. Run M in parallel on all strings in Σ^*
   2. If M accepts any of this strings, *accept*
2. Query the oracle to determine whether $< N, 0 > \in A_{TM}$
3. If the oracle answer No, *accept*; if Yes, *reject*

Turing reducibility

If M's language isn't empty, N will accept every input and, in particular, input 0. Hence the oracle will answer YES, and $T^{A_{TM}}$ will reject. Conversely, if M's language is empty, $T^{A_{TM}}$ will accept. Thus $T^{A_{TM}}$ decides E_{TM}. We say that E_{TM} is **decidable relative** to A_{TM}. That brings us to the definition of Turing reducibility:

Definition

Language A is Turing reducible to language B, written $A \leq_T B$, if A is decidable relative to B

Turing reducibility

Theorem

If $A \leq_T B$ and B is decidable, then A is decidable

Proof.

If B is decidable, then we may replace the oracle for B by an actual procedure that decides B. Thus, we may replace the oracle Turing machine that decides A by an ordinary Turing machine that decides A. □

Turing reducibility

Turing reducibility is a generalization of mapping reducibility. If $A \leq_M B$, then $A \leq_T B$ because the mapping reduction may be used to give an oracle Turing machine that decides A relative to B.

An oracle Turing machine with an oracle for A_{TM} is very powerful. It can solve many problems that are not solvable by ordinary Turing machines. But even such a powerful machine cannot decide all languages